

Capítulo 22

Templates

Exercícios — Resolução Didática

“Resolvemos os 18 exercícios 22.3 a 22.20. Cinco são de programação (22.3 `selectionSort`, 22.4 `printArray` com `range`, 22.5 sobrecarga não-template para `C-strings`, 22.6 `isEqualTo` em três cenários, 22.7 `Array` parametrizado com especialização para `float`); os demais são conceituais, formando o vocabulário completo de templates em `C++`. Todos os códigos compilam com `g++ -Wall -Wextra -std=c++14` sem warnings.”

1 Exercício 22.3 — Template de `selectionSort`

Enunciado. Escreva uma função `selectionSort` usando template. Escreva um programa controlador que insira, classifique e imprima um array de `int` e um array de `float`.

Algoritmo

O *selection sort* é um dos algoritmos mais simples de ordenação. A cada passo:

1. Encontra o menor elemento da parte não-ordenada;
2. Troca-o com o primeiro elemento dessa parte;
3. Repete para a parte restante.

A complexidade é $O(n^2)$ comparações e $O(n)$ trocas (pior que *insertion sort* em arrays quase-ordenados, mas com menos escritas em memória — vantagem em sistemas onde escrita é cara).

Implementação

```
1 // Exercício 22.3 - selectionSort.cpp
2 // Template de selection sort: classifica em ordem CRESCENTE.
3 // Funciona para qualquer tipo T que suporte operator< e operator=.
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 // Template de funcao: T eh o tipo parametrizado
9 template <typename T>
10 void selectionSort(T arr[], int tamanho) {
11     for (int i = 0; i < tamanho - 1; ++i) {
12         // Procura o menor elemento na sub-array [i..tamanho-1]
13         int indiceMenor = i;
14         for (int j = i + 1; j < tamanho; ++j) {
```

```

15         if (arr[j] < arr[indiceMenor])
16             indiceMenor = j;
17     }
18     // Troca arr[i] com arr[indiceMenor]
19     if (indiceMenor != i) {
20         T temp = arr[i];
21         arr[i] = arr[indiceMenor];
22         arr[indiceMenor] = temp;
23     }
24 }
25 }
26
27 // Template auxiliar para imprimir um array
28 template <typename T>
29 void imprimirArray(const T arr[], int tamanho) {
30     for (int i = 0; i < tamanho; ++i)
31         cout << arr[i] << " ";
32     cout << endl;
33 }
34
35 int main() {
36     // === Teste 1: array de int ===
37     cout << "=== Teste com array de int ===" << endl;
38     int vi[] = {64, 25, 12, 22, 11, 90, 1, 33};
39     const int N1 = sizeof(vi) / sizeof(vi[0]);
40     cout << "Antes:   "; imprimirArray(vi, N1);
41     selectionSort(vi, N1);
42     cout << "Depois:  "; imprimirArray(vi, N1);
43
44     // === Teste 2: array de float ===
45     cout << "\n=== Teste com array de float ===" << endl;
46     float vf[] = {3.14f, 1.41f, 2.71f, 0.577f, 1.618f};
47     const int N2 = sizeof(vf) / sizeof(vf[0]);
48     cout << fixed << setprecision(3);
49     cout << "Antes:   "; imprimirArray(vf, N2);
50     selectionSort(vf, N2);
51     cout << "Depois:  "; imprimirArray(vf, N2);
52
53     // === Teste 3 (bonus): array de char ===
54     cout << "\n=== Teste com array de char ===" << endl;
55     char vc[] = {'h', 'e', 'l', 'l', 'o'};
56     const int N3 = sizeof(vc) / sizeof(vc[0]);
57     cout << "Antes:   "; imprimirArray(vc, N3);
58     selectionSort(vc, N3);
59     cout << "Depois:  "; imprimirArray(vc, N3);
60
61     return 0;
62 }

```

Execução

Saída da execução

```

=== Teste com array de int ===
Antes:   64 25 12 22 11 90 1 33
Depois:  1 11 12 22 25 33 64 90

=== Teste com array de float ===
Antes:   3.140 1.410 2.710 0.577 1.618
Depois:  0.577 1.410 1.618 2.710 3.140

```

```
=== Teste com array de char ===  
Antes:  h e l l o  
Depois: e h l l o
```

Discussão

- **O mesmo código, três tipos diferentes.** `selectionSort<int>`, `selectionSort<float>` e `selectionSort<char>` são três especializações geradas pelo compilador. Cada uma vira código de máquina único, otimizado para o tipo.
- **Dedução automática de tipos.** Não é necessário escrever `selectionSort<int>(vi, N1)`; o compilador deduz `T = int` a partir do argumento. Esse é o “*template argument deduction*”.
- **Requisito implícito sobre T.** O template assume que T suporta `operator<` e `operator=`. Tipos que não suportam (como uma classe sem comparação definida) causariam erro de compilação na instanciação.

Observação de Engenharia de Software

Em C++20, o conceito de *Concepts* formalizou esses requisitos implícitos:

```
template <typename T>  
requires std::totally_ordered<T>    // C++20  
void selectionSort(T arr[], int tamanho);
```

Mensagens de erro de compilação tornam-se muito mais claras com Concepts. No nosso `-std=c++14`, ainda dependemos das mensagens crípticas do compilador.

2 Exercício 22.4 — printArray com intervalo (*range*)

Enunciado. Sobrecarregue o template `printArray` para receber também `int lowSubscript` e `int highSubscript`. Valide os índices: retorne 0 se inválidos, senão retorne o número de elementos impressos.

Análise

A nova sobrecarga é também um template, com mais dois parâmetros. As verificações pedidas no enunciado:

- `lowSubscript` ou `highSubscript` fora dos limites → retorna 0;
- `highSubscript ≤ lowSubscript` → retorna 0.

Implementação

Veja o arquivo completo no Exercício 22.5, que combina ambas (e mais a versão não-template). Aqui destacamos só a versão com `range`:

```
template <typename T>
int printArray(const T arr[], int tamanho,
               int lowSubscript, int highSubscript) {
    if (tamanho < 0 || lowSubscript < 0 || highSubscript < 0
        || lowSubscript >= tamanho || highSubscript >= tamanho
        || highSubscript <= lowSubscript) {
        return 0;
    }
    int contador = 0;
    for (int i = lowSubscript; i <= highSubscript; ++i) {
        cout << arr[i] << " ";
        ++contador;
    }
    cout << endl;
    return contador;
}
```

Testes do enunciado

- Intervalo válido (`a[1..3]`): imprime e retorna 3.
- Intervalo *degenerate* (`a[2..2]`): retorna 0 porque `high ≤ low`.
- Intervalo *invertido* (`a[4..1]`): retorna 0.
- `high` fora dos limites (`a[0..10]` num array de 5): retorna 0.

(Veja a saída na execução do Ex. 22.5.)

3 Exercício 22.5 — Sobrecarga não-template para C-strings

Enunciado. Sobrecarregue `printArray` com uma versão *não-template* que imprima um array de strings de caracteres em formato organizado, em colunas.

Análise

Por que precisamos de uma versão especial para C-strings? Porque o template genérico, ao receber `const char*`, imprimiria os ponteiros *como ponteiros* (endereços de memória), não como strings, em razão de uma sutileza de C: arrays de `char*` não se “desreferenciam” automaticamente.

(Na verdade, com `std::cout`, `char*` já se converte para a string apontada por uma sobrecarga existente do `operator«` — mas isso não nos ajuda a obter *colunas alinhadas*. A sobrecarga não-template oferece exatamente esse controle.)

Implementação completa (junta os 3 `printArray`)

```

1  // Exercicios 22.4 e 22.5 - printArray.cpp
2  // Tres versoes coexistem:
3  // 1. Template original (Fig. 22.1 do livro)
4  // 2. Template SOBRECARREGADO com (lowSubscript, highSubscript) [22.4]
5  // 3. Versao NAO-TEMPLATE para const char* [22.5]
6  #include <iostream>
7  #include <iomanip>
8  using namespace std;
9
10 // === Versao 1 (original do livro): imprime array inteiro ===
11 template <typename T>
12 void printArray(const T arr[], int tamanho) {
13     for (int i = 0; i < tamanho; ++i)
14         cout << arr[i] << " ";
15     cout << endl;
16 }
17
18 // === Versao 2 (Exercicio 22.4): imprime subintervalo ===
19 // Retorna o numero de elementos impressos, ou 0 em caso de erro.
20 template <typename T>
21 int printArray(const T arr[], int tamanho,
22               int lowSubscript, int highSubscript) {
23     if (tamanho < 0 || lowSubscript < 0 || highSubscript < 0
24         || lowSubscript >= tamanho || highSubscript >= tamanho
25         || highSubscript <= lowSubscript) {
26         return 0;
27     }
28     int contador = 0;
29     for (int i = lowSubscript; i <= highSubscript; ++i) {
30         cout << arr[i] << " ";
31         ++contador;
32     }
33     cout << endl;
34     return contador;
35 }
36
37 // === Versao 3 (Exercicio 22.5): nao-template para const char* ===
38 // (Especializacao explicita por sobrecarga, n\~ao por template<>.)
39 // Imprime strings em formato organizado, em colunas alinhadas.
40 void printArray(const char *const arr[], int tamanho) {
41     const int LARGURA_COLUNA = 15;
42     const int COLUNAS_POR_LINHA = 4;
43     for (int i = 0; i < tamanho; ++i) {
44         cout << setw(LARGURA_COLUNA) << left << arr[i];
45         if ((i + 1) % COLUNAS_POR_LINHA == 0) cout << endl;
46     }
47     if (tamanho % COLUNAS_POR_LINHA != 0) cout << endl;
48 }
49
50 int main() {
51     // Arrays do livro (figura 22.1)

```

```

52  int    a[] = {1, 2, 3, 4, 5};
53  float  b[] = {1.1f, 2.2f, 3.3f, 4.4f, 5.5f};
54  char   c[] = "OLA";           // string como array de char
55
56  // === V1: imprimir arrays inteiros ===
57  cout << "=== printArray (versao original): ===" << endl;
58  cout << "a: "; printArray(a, 5);
59  cout << "b: "; printArray(b, 5);
60  cout << "c: "; printArray(c, 3);    // imprime 'O' 'L' 'A'
61
62  // === V2: imprimir subintervalos (Ex. 22.4) ===
63  cout << "\n=== printArray com range (Ex. 22.4): ===" << endl;
64  cout << "a[1..3]: ";
65  int n = printArray(a, 5, 1, 3);
66  cout << " (imprimiu " << n << " elementos)" << endl;
67
68  cout << "b[0..4]: ";
69  n = printArray(b, 5, 0, 4);
70  cout << " (imprimiu " << n << " elementos)" << endl;
71
72  // Testes de validacao:
73  cout << "Tentativa intervalo invalido (low=4 > high=1):" << endl;
74  n = printArray(a, 5, 4, 1);
75  cout << " retornou " << n << " (sem impressao)" << endl;
76
77  cout << "Tentativa fora dos limites (high=10):" << endl;
78  n = printArray(a, 5, 0, 10);
79  cout << " retornou " << n << " (sem impressao)" << endl;
80
81  cout << "Tentativa low = high (degenerado):" << endl;
82  n = printArray(a, 5, 2, 2);
83  cout << " retornou " << n << " (sem impressao porque high <= low)" << endl;
84
85  // === V3: array de strings (Ex. 22.5) ===
86  cout << "\n=== printArray para strings (Ex. 22.5): ===" << endl;
87  const char *nomes[] = {
88      "Alice",    "Bob",    "Carlos",    "Daniela",
89      "Eduardo",  "Fernanda", "Gabriel",  "Helena",
90      "Igor",     "Julia"
91  };
92  printArray(nomes, 10);
93
94  return 0;
95  }

```

Execução (completa, cobrindo Ex. 22.4 e 22.5)

Saída da execução

```

=== printArray (versao original): ===
a: 1 2 3 4 5
b: 1.1 2.2 3.3 4.4 5.5
c: O L A

=== printArray com range (Ex. 22.4): ===
a[1..3]: 2 3 4
(imprimiu 3 elementos)
b[0..4]: 1.1 2.2 3.3 4.4 5.5
(imprimiu 5 elementos)
Tentativa intervalo invalido (low=4 > high=1):
retornou 0 (sem impressao)

```

```

Tentativa fora dos limites (high=10):
    retornou 0 (sem impressao)
Tentativa low = high (degenerado):
    retornou 0 (sem impressao porque high <= low)

=== printArray para strings (Ex. 22.5): ===
Alice          Bob          Carlos          Daniela
Eduardo        Fernanda    Gabriel        Helena
Igor           Julia

```

Discussão — precedence de sobrecarga em C++

Quando o compilador encontra uma chamada como `printArray(nomes, 10)` com `nomes` sendo `const char*[]`, ele:

1. Primeiro busca funções *não-template* que se encaixem.
2. Só se nenhuma se encaixar, considera templates.

Como temos `void printArray(const char *const arr[], int)` (não-template), ela “ganha” do template genérico — por isso a formatação em colunas é usada.

Boa Prática de Programação

Quando preferir não-template a especialização explícita: a versão não-template é mais simples de escrever e tem regras de resolução de overload mais previsíveis. Especializações explícitas (`template<>`) têm sutilezas em C++ que muitas vezes surpreendem programadores. Por essa razão, comunidades como ISO C++ Core Guidelines recomendam *evitar* especializações explícitas de templates de função — prefira sobrecarga.

4 Exercício 22.6 — isEqualTo em três cenários

Enunciado. Escreva um template simples `isEqualTo` que compare dois argumentos com `==`. Use com tipos fundamentais (**cenário A**). Depois, escreva uma versão que use uma classe sem `operator==` (**cenário B**) — o que acontece? Finalmente, sobrecarregue `operator==` (**cenário C**) — o que acontece agora?

Template

Em todos os três cenários, o template é o mesmo:

```
template <typename T>
bool isEqualTo(const T &a, const T &b) {
    return a == b;
}
```

Cenário A — tipos fundamentais (sucesso)

```
1 // Exercicio 22.6 (Cenario A) - isEqualTo_primitivos.cpp
2 // Template isEqualTo testado com tipos primitivos
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 template <typename T>
8 bool isEqualTo(const T &a, const T &b) {
9     return a == b;
10 }
11
12 int main() {
13     cout << boolalpha;
14
15     cout << "=== Testes com tipos primitivos ===" << endl;
16     cout << "isEqualTo(10, 10)      = " << isEqualTo(10, 10) << endl;
17     cout << "isEqualTo(10, 20)      = " << isEqualTo(10, 20) << endl;
18     cout << "isEqualTo(3.14, 3.14)   = " << isEqualTo(3.14, 3.14) << endl;
19     cout << "isEqualTo('a', 'a')      = " << isEqualTo('a', 'a') << endl;
20     cout << "isEqualTo('a', 'b')      = " << isEqualTo('a', 'b') << endl;
21
22     // string da biblioteca padrao tem operator==
23     string s1 = "Ola";
24     string s2 = "Ola";
25     string s3 = "Mundo";
26     cout << "isEqualTo(s1, s2)      = " << isEqualTo(s1, s2) << endl;
27     cout << "isEqualTo(s1, s3)      = " << isEqualTo(s1, s3) << endl;
28
29     return 0;
30 }
```

Saída da execução

```
=== Testes com tipos primitivos ===
isEqualTo(10, 10)      = true
isEqualTo(10, 20)      = false
isEqualTo(3.14, 3.14)   = true
isEqualTo('a', 'a')     = true
isEqualTo('a', 'b')     = false
isEqualTo(s1, s2)       = true
```



```
isEqualTo(s1, s3) = false
```

Cenário B — classe SEM operator== (falha de compilação)

```

1 // Exercício 22.6 (Cenário B) - isEqualTo_classeSem.cpp
2 // Mostra que SEM operator==, a classe NAO COMPILA com isEqualTo.
3 //
4 // Para reproduzir o erro do livro, basta DESCOMENTAR a linha indicada.
5 // O código abaixo permanece compilável sem ele (so referencia o tipo).
6
7 #include <iostream>
8 #include <string>
9 using namespace std;
10
11 class Ponto {
12 public:
13     Ponto(double x = 0, double y = 0) : x(x), y(y) {}
14     // SEM operator==
15 private:
16     double x, y;
17 };
18
19 template <typename T>
20 bool isEqualTo(const T &a, const T &b) {
21     return a == b;    // chamara operator== de T
22 }
23
24 int main() {
25     Ponto p1(1.0, 2.0);
26     Ponto p2(1.0, 2.0);
27
28     // A linha abaixo NAO COMPILA porque Ponto nao define operator==
29     // Para reproduzir o erro: descomente a linha:
30     //
31     // cout << isEqualTo(p1, p2) << endl;
32     //
33     // Mensagem do g++:
34     // error: no match for 'operator==' (operand types are 'const Ponto'
35     // and 'const Ponto')
36     // note: in instantiation of 'bool isEqualTo(const T&, const T&)'
37     // [with T = Ponto]
38
39     cout << "(Cenario B - compila apenas porque a chamada esta comentada.)\n";
40     cout << "Pontos criados; sem operator==, isEqualTo nao pode instanciar.\n";
41
42     // Para evitar warning de variavel nao usada:
43     (void)p1; (void)p2;
44     return 0;
45 }
```

Quando descomentamos a linha `cout << isEqualTo(p1, p2)`, o compilador gera:

Saída da execução

```

isEqualTo_classeSem.cpp:9:51: error: no match for 'operator=='
      (operand types are 'const Ponto' and 'const Ponto')
      9 | bool isEqualTo(const T &a, const T &b) { return a == b; }
        |                                     ~~~~^~~~
note: in instantiation of 'bool isEqualTo(const T&, const T&)'
      [with T = Ponto]
```

O programa *não compila*. A mensagem identifica:

- Onde falhou: `return a == b`.
- Por quê: “no match for operator==”.
- Contexto: “in instantiation of `isEqualTo...` with `T = Ponto`”.

Cenário C — classe COM `operator==` (sucesso)

```

1  // Exercício 22.6 (Cenário C) - isEqualTo_classeCom.cpp
2  // AGORA com operator== sobrecarregado, isEqualTo COMPILA E FUNCIONA.
3
4  #include <iostream>
5  using namespace std;
6
7  class Ponto {
8  public:
9      Ponto(double x = 0, double y = 0) : x(x), y(y) {}
10
11     // Sobrecarga de operator==
12     bool operator==(const Ponto &outro) const {
13         return x == outro.x && y == outro.y;
14     }
15 private:
16     double x, y;
17 };
18
19 template <typename T>
20 bool isEqualTo(const T &a, const T &b) {
21     return a == b;
22 }
23
24 int main() {
25     cout << boolalpha;
26
27     Ponto p1(1.0, 2.0);
28     Ponto p2(1.0, 2.0);
29     Ponto p3(3.0, 4.0);
30
31     cout << "=== Cenário C: classe Ponto AGORA com operator== ===" << endl;
32     cout << "isEqualTo(p1, p2) = " << isEqualTo(p1, p2)
33         << "    (esperado true: ambos (1,2))" << endl;
34     cout << "isEqualTo(p1, p3) = " << isEqualTo(p1, p3)
35         << "    (esperado false)" << endl;
36     cout << "isEqualTo(p2, p2) = " << isEqualTo(p2, p2)
37         << "    (esperado true)" << endl;
38
39     return 0;
40 }

```

Saída da execução

```

=== Cenário C: classe Ponto AGORA com operator== ===
isEqualTo(p1, p2) = true    (esperado true: ambos (1,2))
isEqualTo(p1, p3) = false   (esperado false)
isEqualTo(p2, p2) = true    (esperado true)

```

Discussão — requisitos implícitos em templates

Esses três cenários ilustram a **característica mais sutil de templates**: eles impõem *requisitos implícitos* sobre o tipo T. `isEqualTo` requer que T suporte `operator==`. Esse requisito só aparece *na hora da instanciã*o.

- **Vantagem:** é “duck typing” em tempo de compilação. Qualquer tipo que “quack” como esperado funciona.
- **Desvantagem:** mensagens de erro podem ser longíssimas e crípticas (especialmente em código da STL).

Observação de Engenharia de Software

A introdução de *Concepts* em C++20 resolve esse problema permitindo declarar requisitos explicitamente:

```
template <std::equality_comparable T>
bool isEqualTo(const T &a, const T &b) { return a == b; }
```

Agora, a mensagem de erro fica clara: “Ponto does not satisfy `std::equality_comparable`”.

5 Exercício 22.7 — Template Array com especialização

Enunciado. Use `template<int numeroDeElementos, typename tipoElemento>` para criar um template `Array`. Sobreponha com uma definição específica para `Array<float>`. Mostre instanciación de `Array` de `int` pelo template, e que `Array` de `float` usa a definição específica.

Análise do problema

Aqui aparecem dois recursos especialmente interessantes:

1. **Parâmetro não-tipo:** `int numeroDeElementos` é um *valor* (não um tipo). O tamanho do array é fixado *em tempo de compilação*.
2. **Especialização parcial:** `Array<float, N>` tem comportamento distinto do template genérico.

Cabeçalho `Array.h`

```

1 // Exercício 22.7 - Array.h
2 // Template de classe Array com DOIS parametros:
3 //   - numeroDeElementos: parametro NAO-TIPO (int conhecido em compilacao)
4 //   - tipoElemento:      parametro de tipo
5 // Inclui especializacao EXPLICITA para Array<float, N>.
6 #ifndef ARRAY_TEMPLATE_H
7 #define ARRAY_TEMPLATE_H
8
9 #include <iostream>
10 #include <iomanip>
11
12 // === Template primario (geral) ===
13 template <typename tipoElemento, int numeroDeElementos>
14 class Array {
15 public:
16     Array() {
17         // Inicializa todos com valor "zero" do tipo
18         for (int i = 0; i < numeroDeElementos; ++i)
19             dados[i] = tipoElemento();
20     }
21
22     int tamanho() const { return numeroDeElementos; }
23
24     tipoElemento &operator[](int i) { return dados[i]; }
25     const tipoElemento &operator[](int i) const { return dados[i]; }
26
27     void imprimir() const {
28         std::cout << "[Array geral] ";
29         for (int i = 0; i < numeroDeElementos; ++i)
30             std::cout << dados[i] << " ";
31         std::cout << std::endl;
32     }
33
34 private:
35     tipoElemento dados[numeroDeElementos];
36 };
37
38 // === Especializacao para float ===
39 // Imprime com precisao fixa de 3 casas, formato fechado entre colchetes.
40 template <int numeroDeElementos>
41 class Array<float, numeroDeElementos> {
42 public:

```

```

43     Array() {
44         for (int i = 0; i < numeroDeElementos; ++i)
45             dados[i] = 0.0f;
46     }
47
48     int tamanho() const { return numeroDeElementos; }
49
50     float &operator[](int i) { return dados[i]; }
51     const float &operator[](int i) const { return dados[i]; }
52
53     void imprimir() const {
54         std::cout << "[Array<float> especializado] [ ";
55         for (int i = 0; i < numeroDeElementos; ++i) {
56             std::cout << std::fixed << std::setprecision(3)
57                 << dados[i];
58             if (i < numeroDeElementos - 1) std::cout << ", ";
59         }
60         std::cout << " ]" << std::endl;
61     }
62
63 private:
64     float dados[numeroDeElementos];
65 };
66
67 #endif

```

Programa de teste

```

1  // Exercício 22.7 - main_Array.cpp
2  #include <iostream>
3  #include "Array.h"
4  using namespace std;
5
6  int main() {
7      // == Instanciação de Array<int, 5> -> usa o template GERAL ==
8      cout << "=== Array<int, 5> (usa template geral) ===" << endl;
9      Array<int, 5> arrInt;
10     for (int i = 0; i < arrInt.tamanho(); ++i)
11         arrInt[i] = (i + 1) * 10; // 10, 20, 30, 40, 50
12     arrInt.imprimir();
13
14     // == Instanciação de Array<double, 3> -> também geral ==
15     cout << "\n=== Array<double, 3> (usa template geral) ===" << endl;
16     Array<double, 3> arrDbl;
17     arrDbl[0] = 1.5;
18     arrDbl[1] = 2.5;
19     arrDbl[2] = 3.5;
20     arrDbl.imprimir();
21
22     // == Instanciação de Array<float, 4> -> usa a ESPECIALIZACAO ==
23     cout << "\n=== Array<float, 4> (usa ESPECIALIZACAO!) ===" << endl;
24     Array<float, 4> arrFloat;
25     arrFloat[0] = 3.14159f;
26     arrFloat[1] = 2.71828f;
27     arrFloat[2] = 1.41421f;
28     arrFloat[3] = 0.57721f;
29     arrFloat.imprimir();
30
31     // == Inspeccionando: o tamanho é conhecido em tempo de COMPILAÇÃO ==
32     cout << "\nTamanho de cada array:" << endl;
33     cout << "    arrInt:    " << arrInt.tamanho() << " elementos" << endl;
34     cout << "    arrDbl:    " << arrDbl.tamanho() << " elementos" << endl;

```

```

35     cout << "   arrFloat: " << arrFloat.tamanho() << " elementos" << endl;
36
37     return 0;
38 }

```

Execução

Saída da execução

```

=== Array<int, 5> (usa template geral) ===
[Array geral] 10 20 30 40 50

=== Array<double, 3> (usa template geral) ===
[Array geral] 1.5 2.5 3.5

=== Array<float, 4> (usa ESPECIALIZACAO!) ===
[Array<float> especializado] [ 3.142, 2.718, 1.414, 0.577 ]

Tamanho de cada array:
  arrInt:    5 elementos
  arrDbl:    3 elementos
  arrFloat:  4 elementos

```

Discussão — por que parâmetro não-tipo?

O tamanho fica embutido no tipo: `Array<int, 5>` e `Array<int, 10>` são *tipos diferentes*. O compilador pode:

- Alocar o tamanho em *tempo de compilação* (sem `new`).
- Detectar erros de *tamanho* em tempo de compilação (não é possível atribuir `Array<int, 5>` a `Array<int, 10>`).
- Aplicar otimizações baseadas no tamanho conhecido (unroll de loops, alinhamento, etc.).

Compare com a abordagem `std::vector<int>`, em que o tamanho é dinâmico (e o array vive no heap). Ambas as abordagens têm valor; `std::array<int, N>` da STL combina *template-tamanho* com a interface moderna.

Sobre a sintaxe `template<int N> class Array<float, N>`

A especialização *não* é do tipo “`template<>`” (*full specialization*), e sim parcial: especializamos só o tipo (para `float`), mantendo o tamanho `N` ainda como parâmetro. C++ permite essa especialização parcial em templates de classe (mas não em templates de função, no qual usaríamos sobrecarga, como no 22.5).

Observação de Engenharia de Software

Diferenças técnicas entre as duas versões do nosso `Array`:

- Versão geral: `imprimir()` chama `cout << dados[i]`, com o formato default.
- Versão especializada para `float`: usa `fixed + setprecision(3)` para mostrar exatamente 3 casas decimais, e formato `[ a, b, c ]` mais visual.

A escolha de *como* se diferenciar é do programador. O importante é que a especialização seja usada *automaticamente* pelo compilador na presença do tipo certo.

6 Exercício 22.8 — Template de função vs especialização

Enunciado. Faça a distinção entre *template de função* e *especialização de template de função*.

Definições

Template de função: é a *forma genérica* com parâmetros de tipo. Não é código executável por si só — é uma *receita* que o compilador segue para gerar funções concretas.

Especialização de template de função: é uma *função concreta* gerada a partir do template para tipos específicos.

Comparação

Aspecto	Template	Especialização
É código executável?	NÃO (só uma receita)	SIM (compilada e executável)
Quantas existem?	Uma definição	Várias (uma por tipo usado)
Quando é gerada?	—	Em compilação, sob demanda
Exemplo de sintaxe	<code>template <typename T> T menor(T a, T b);</code>	(Implícita ao usar:) <code>menor<int>(...)</code>

Analogia

Pense em um template como um *molde para biscoitos*, e cada especialização como um *biscoito* feito desse molde, mas com sabor próprio (chocolate, baunilha, manteiga ...). O molde não se come; os biscoitos sim.

7 Exercício 22.9 — Template como estêncil

Enunciado. O que é mais semelhante a um estêncil: um template de classe ou uma especialização de template de classe?

Resposta: Um template de classe é mais semelhante a um estêncil.

Justificativa

Um *estêncil* é um molde vazado: você pinta sobre ele com uma cor (parâmetro) e obtém uma figura concreta. **Não se exhibe o estêncil** na parede; **exibe-se as figuras pintadas através dele**.

Analogamente:

- **Template de classe** (o “estêncil”): `template <typename T> class Array`.
- **Especializações** (as “figuras pintadas”): `Array<int>`, `Array<double>`, `Array<Forma>`.

O programa só usa as especializações (objetos concretos com tipo conhecido); o template em si é um construto auxiliar do compilador.

8 Exercício 22.10 — Templates e sobrecarga

Enunciado. Qual é a relação entre templates de função e sobrecarga?

Três níveis de relação

1. Templates *geram* funções sobrecarregadas

Cada especialização gerada (`selectionSort<int>`, `selectionSort<float>`, ...) é uma função distinta, com o mesmo nome. É sobrecarga “*de graça*” pelo template.

2. Templates podem *coexistir* com funções não-template

Foi o caso do nosso 22.5: `printArray` tem (a) o template genérico e (b) a versão para `const char**`. Quando o compilador encontra `printArray(nomes, 10)`, escolhe entre ambas pelas regras de sobrecarga.

3. Templates podem *sobrecarregar entre si*

Templates podem coexistir se diferem em número ou tipo de parâmetros. Foi o caso do 22.4: `printArray(arr, n)` e `printArray(arr, n, low, high)` são templates distintos.

Regra de preferência na resolução

Quando o compilador tem opções em todos os três níveis, ele segue a **regra do mais específico**:

1. **Mais específico:** função não-template com tipo exato.
2. **Especialização explícita** de um template (`template<>`).
3. **Template genérico** (instanciado para o tipo).

9 Exercício 22.11 — Template vs macro

Enunciado. Por que você usaria um template de função no lugar de uma macro?

O problema de macros

Macros (`#define`) são substituições textuais cegas, processadas pelo *pré-processador* antes da compilação. Famosa armadilha:

```
#define QUADRADO(x) x * x

int y = QUADRADO(2 + 3);
// expande para: 2 + 3 * 2 + 3 = 11 (deveria ser 25!)
```

A solução parcial (parênteses obsessivos) é:

```
#define QUADRADO(x) ((x) * (x))
```

Mas isso ainda tem problemas:

```
int y = QUADRADO(++i);
// expande para: ((++i) * (++i))  <-- comportamento indefinido!
```


Vantagens dos templates sobre macros

Característica	Macro	Template
Verificação de tipos	NÃO	SIM
Avaliação consistente de argumentos	NÃO (pode duplicar efeitos)	SIM
Mensagens de erro com contexto	NÃO (referem-se ao código expandido)	SIM (referem-se ao código ori
Compatível com <i>debuggers</i>	RUIM	BOM
Acesso a membros privados?	NÃO	SIM (se <i>friend</i>)

Exemplo equivalente com template

```
template <typename T>
T quadrado(T x) {
    return x * x;
}

int y = quadrado(++i);    // ++i avaliado UMA vez; comportamento bem definido
```

Boa Prática de Programação

Em C++ moderno, evite macros para definir funções ou constantes. Use `template` para funções genéricas, `constexpr` para constantes computadas, e `inline` para sugestões de expansão em linha. As únicas situações em que macros ainda fazem sentido são guardas de inclusão (`#ifndef/#define/#endif`) e configuração condicional de compilação.

10 Exercício 22.12 — Problema de desempenho com templates

Enunciado. Que problema de desempenho pode resultar do uso de templates?

O problema: code bloat

Cada especialização usada gera código distinto. Se um programa usa `std::vector<int>`, `std::vector<double>`, `std::vector<string>`, `std::vector<Funcionario>`, ..., o compilador gera 4 versões completas de cada método do `vector`. Isso pode resultar em:

- **Binário maior** (megabytes a mais em casos extremos).
- **Cache de instruções mais poluído**, que paradoxalmente pode *piorar* o desempenho geral.
- **Tempo de compilação longo** (templates são notoriamente lentos de compilar).

Como mitigar

1. **Fatorar código** para retirar do template tudo que não depende de `T` (chamando uma função auxiliar não-template).
2. **Compilação precompiled headers** (cabeçalhos pré-compilados) para acelerar a compilação.
3. **Usar `extern template`** (C++11) para evitar gerar a especialização em múltiplos arquivos.
4. **type erasure** para problemas em que o overhead de virtuais é aceitável mas o bloat não é (ex.: `std::function`).

Dica de Desempenho

A bibliotecas STL e Boost são notórias por usar templates pesadamente. Compilar projetos que usam Boost.Spirit ou Eigen pode levar minutos por arquivo. É o trade-off clássico: rapidez em tempo de execução vs. rapidez em tempo de compilação.

11 Exercício 22.13 — Quando a correspondência falha

Enunciado. O compilador realiza correspondência para escolher qual especialização chamar. Sob que circunstâncias isso resulta em erro de compilação?

Três categorias de falha

1. Não existe especialização compatível com os argumentos

```
template <typename T> T menor(T a, T b);

// Chamada:
menor(5, "ola");           // ERRO: T nao pode ser int E string simultaneamente
```

2. Os requisitos implícitos sobre `T` não são satisfeitos

Exatamente o cenário B do nosso 22.6: a classe `Ponto` sem `operator==` causa erro ao instanciar `isEqualTo<Ponto>`.

3. Ambigüidade entre múltiplas especializações/sobrecargas

```
template <typename T>          void f(T x);
template <typename T>          void f(T *x);

int a = 5;
f(&a);    // AMBIGUO: poderia ser f<int*>(int*) ou f<int>(int*)
```

Estratégia para evitar

- Em C++20: *Concepts* para tornar requisitos explícitos.
- Em C++14: `static_assert` dentro do template:

```
template <typename T>
void selectionSort(T arr[], int n) {
    static_assert(std::is_default_constructible<T>::value,
                  "T precisa ser default-constructible");
    // ...
}
```

12 Exercício 22.14 — Tipo parametrizado

Enunciado. Por que é apropriado referir-se a um template de classe como um “tipo parametrizado”?

Resposta: porque um template de classe *descreve uma família de tipos*, indexada por parâmetros. Cada combinação de parâmetros gera um *tipo* distinto.

Exemplificando

```
template <typename T, int N> class Array;

Array<int, 5>      a;    // TIPO 1
Array<int, 10>     b;    // TIPO 2 (distinto do Tipo 1, ainda que parecido)
Array<double, 5>   c;    // TIPO 3
Array<Forma*, 100> d;    // TIPO 4
```

A frase “`Array<int, 5>`” é o nome *completo* do tipo. O compilador trata `Array<int, 5>` e `Array<int, 10>` como tipos *não-relacionados*.

```
Array<int, 5> a;
Array<int, 10> b;
a = b;    // ERRO: tipos diferentes!
```

13 Exercício 22.15 — `Array<Funcionario> lista(100)`

Enunciado. Explique por que um programa em C++ usaria `Array<Funcionario> listaTrabalhador(100);`

Decifrando a instrução

- `Array` é um template de classe.
- `<Funcionario>` especifica o parâmetro de tipo: o array conterá objetos `Funcionario`.
- `listaTrabalhador` é o nome do objeto criado.
- `(100)` é argumento para o construtor: especifica o tamanho do array (note: *não* é parâmetro do template, mas do construtor!).

Por que usar isso?

- Você precisa de uma coleção homogênea de funcionários.
- O tamanho *máximo* é conhecido em runtime (não em compilação), por isso o argumento `100` no construtor.
- Você quer benefícios do encapsulamento da classe `Array` (provavelmente verificação de limites, atribuição com checagem de tamanho, etc.).

14 Exercício 22.16 — `Array<Funcionario> lista;`

Enunciado. Por que um programa usaria `Array<Funcionario> listaTrabalhador;`?

Diferença em relação a 22.15

A única diferença: *não* há argumento para o construtor. Se a classe `Array` fornece um construtor *default* (sem argumentos), `listaTrabalhador` será criado com tamanho default — talvez 0, talvez algum tamanho razoável (10, por exemplo, como em muitas implementações didáticas).

Por que você faria isso?

- Tamanho desconhecido no momento da declaração — vai crescer dinamicamente.
- Confia no comportamento default da classe.
- É mais conciso quando não é preciso especificar tamanho.

Em conjunto com 22.15

A combinação mostra um caso comum: uma classe `Array` oferece vários construtores (com e sem argumento). Esse é o padrão de fato em contêineres padrão (`std::vector`):

```
std::vector<Funcionario> a;           // vazio, capacidade inicial 0
std::vector<Funcionario> b(100);      // 100 funcionarios default
std::vector<Funcionario> c(100, f);   // 100 funcionarios copia de f
```

15 Exercício 22.17 — Sintaxe `Array<T>::Array(int s)`

Enunciado. Explique o uso da seguinte notação:

```
template< typename T > Array< T >::Array ( int s )
```

Decifrando linha por linha

```
template <typename T>           // (1) Cabeçalho: somos um template parametrizado por
    T
Array<T>::                      // (2) Definimos uma funcao da classe Array<T>
Array(int s)                   // (3) Especificamente, o construtor Array(int s)
{
    // implementacao do construtor
}
```

Contexto: definição fora da classe

Isto aparece quando o programador separa *declaração* e *definição* em arquivos diferentes (ou simplesmente quer manter o corpo dos métodos fora da declaração da classe):

```
// No header (.h):
template <typename T>
class Array {
public:
    Array(int s);                // <-- so' declaracao
    // ...
};

// Em outro lugar (mesmo .h, ou .cpp se nao fosse template):
template <typename T>
Array<T>::Array(int s) {        // <-- definicao
    // ...
}
```

Pontos sutis

- O cabeçalho `template <typename T>` se repete (cada função-membro definida fora precisa do seu próprio cabeçalho).
- Sob a sintaxe `Array<T>::Array`, o segundo `Array` é o nome do construtor (sem tipo de retorno, como qualquer construtor).
- Definição em `.cpp`? Geralmente não funciona com templates. Como o compilador precisa do código no ponto de instanciação, geralmente colocamos templates inteiramente em arquivos `.h`.

Erro Comum de Programação

Tentar separar template entre `.h` e `.cpp` é uma fonte clássica de erros do tipo “undefined reference”. A solução mais simples é manter todo o template no header (eventualmente em um `.hpp` incluído pelo `.h`).

16 Exercício 22.18 — Por que parâmetros não-tipo em contêiners?

Enunciado. Por que você usaria um parâmetro não-tipo com um template de classe para um contêiner como um array ou uma pilha?

Três vantagens de tamanho como parâmetro

1. Alocação na pilha

```
template <typename T, int N> class Array {
    T dados[N];           // alocado NA PILHA
};

Array<int, 100> a;        // 400 bytes na pilha; nada de malloc/new
```

Compare com `std::vector<int>` que aloca dinamicamente. Em código de baixíssimo overhead (sistemas embarcados, código crítico), alocação na pilha é drasticamente mais rápida que no heap.

2. Verificação de limites em compilação

```
template <typename T, int N> class Array {
    template <int I>
    T &get() {
        static_assert(I < N, "índice fora dos limites");
        return dados[I];
    }
};

Array<int, 5> a;
a.get<2>();           // OK
a.get<10>();          // erro DE COMPILAÇÃO!
```

3. Tipos distintos para tamanhos distintos

`Array<int, 5>` e `Array<int, 10>` são tipos *diferentes*. Impossível atribuir um ao outro acidentalmente.

Quando *não* usar parâmetro de tamanho

Quando o tamanho varia em runtime — `std::vector` faz exatamente isso. Cada paradigma serve a um cenário distinto.

17 Exercício 22.19 — Relações de friend

Enunciado. O template `template< typename T > class Ct1` contém uma das declarações `friend` abaixo. Descreva a relação estabelecida.

(a) `friend void f1();`

Declara que a *função livre* `f1` (não-template, sem parâmetros) é amiga de *todas* as especializações de `Ct1`. Ou seja, `f1` pode acessar membros privados de `Ct1<int>`, `Ct1<double>`, `Ct1<Funcionario>`, etc.

(b) `friend void f2(Ct1<T>&);`

Declara que `f2`, que aceita uma referência a `Ct1<T>`, é amiga da *especialização corrente* (a do mesmo `T`). Isto é, `f2(Ct1<int>&)` é amiga de `Ct1<int>`; `f2(Ct1<double>&)` é amiga de `Ct1<double>`.

(c) `friend void C2::f3();`

Declara que o *método* `f3` da *classe* `C2` (não-template) é amigo de `Ct1`. Apenas esse método, não o restante de `C2`.

(d) `friend void Ct3<T>::f4(Ct1<T>&);`

Declara que o método `f4` da classe `Ct3<T>` (para o mesmo `T`) é amigo da `Ct1<T>` correspondente. Exemplo: `Ct3<int>::f4` é amigo de `Ct1<int>`.

(e) `friend da classe C4;`

(Sintaxe completa: `friend class C4;`) Declara que *toda* a classe `C4` (não-template) é amiga — todos os seus métodos podem acessar membros privados de `Ct1`.

(f) `friend da classe Ct5<T>;`

(Sintaxe completa: `friend class Ct5<T>;`) Declara que a *especialização* `Ct5<T>` (para o mesmo `T`) é amiga de `Ct1<T>`.

18 Exercício 22.20 — static em template de classe

Enunciado. Suponha que `Funcionario` (template de classe) tenha um dado-membro `static contador`. Três especializações do template são instanciadas. Quantas cópias do `static` existirão? Como o acesso a cada uma é restrito?

Resposta

Três cópias — uma para cada especialização. Cada especialização é um *tipo distinto*, então cada uma tem sua própria *variável estática*.

Exemplo explícito

```
template <typename T>
class Funcionario {
public:
    static int contador;
};

template <typename T>
int Funcionario<T>::contador = 0;    // definicao no namespace global

//Codigo cliente:
Funcionario<int>::contador          = 5;    // contador SO de Funcionario<int>
Funcionario<double>::contador      = 10;    // contador SO de Funcionario<double>
Funcionario<string>::contador      = 15;    // contador SO de Funcionario<string>

cout << Funcionario<int>::contador    << endl;    // 5
cout << Funcionario<double>::contador << endl;    // 10
cout << Funcionario<string>::contador << endl;    // 15
```

Restrição de acesso

Cada `contador` é acessível apenas pelo nome qualificado completo (`Funcionario<TIPO>::contador`). Não se pode acessar “`Funcionario::contador`” (sem `<TIPO>`); o compilador exige que você especifique de qual especialização quer falar.

Observação de Engenharia de Software

Essa é uma das diferenças mais sutis de templates: comportam-se como *tipos distintos* para todos os efeitos. Inclusive: `Funcionario<int>` e `Funcionario<double>` podem ter inclusive métodos com assinaturas diferentes (especializações podem reescrever a interface). Não existe um “`Funcionario`” genérico — existe um template genérico, mas não um *tipo* genérico.

Encerramento

Os 18 exercícios deste capítulo cobriram todo o território de templates em C++:

- **Templates de função** com dedução de tipos (22.3 `selectionSort`; 22.4 `printArray` com `range`).
- **Sobrecarga de templates com funções não-template** (22.5).
- **Requisitos implícitos sobre T** (22.6).
- **Templates de classe com parâmetros mistos** (tipo + não-tipo) (22.7).
- **Especialização parcial** (22.7).
- **Conceito de tipo parametrizado, estêncil, biscoito** (22.8, 22.9, 22.14).
- **Templates vs macros, vs sobrecarga** (22.10, 22.11).
- **Code bloat e desempenho** (22.12).
- **Resolução de overload e ambigüidades** (22.13).
- **Sintaxe de definição externa, parâmetros não-tipo, friends, statics** (22.15–22.20).

Templates são o *polimorfismo estático* de C++ — um complemento poderoso ao polimorfismo dinâmico do Cap. 21. Em conjunto, oferecem soluções elegantes para a maioria dos problemas de generalidade em sistemas reais. O Capítulo 23 cobrirá **tratamento de exceções**, a técnica fundamental para lidar com erros em código genérico.